

# Taller: Control de Calidad de Software

Unidad 2 - Servicios APIs





- Historia de los Servicios API
- Definición de Servicios API
- Tipos de Servicios
- Diferencia entre Servicios API REST y SOAP
- Métodos API REST
  - GET
  - POST
  - PUT
  - DELETE
- ¿Por qué probar APIs o servicios?



Los servicios API existieron durante décadas en diversas formas, pero su uso se volvió una necesidad con el crecimiento de Internet y la necesidad de que las aplicaciones se comuniquen entre sí de manera más eficiente y flexible



SOAP (Simple Object Access Protocol)



REST (Representational State Transfer)  
2004 / 2006 se implementa RESTful

## Los 4 niveles de Madurez de Richardson RESTful

Para que una API se gane el título de RESTful, suele medirse con el Modelo de Madurez de Richardson:

- Nivel 0 (The Swamp of POX): Usamos HTTP solo como transporte (como un túnel), pero no aprovechamos sus ventajas.
- Nivel 1 (Recursos): Usamos URLs individuales para cada recurso (ej. /usuarios/1).
- Nivel 2 (Verbos HTTP): Usamos GET para leer, POST para crear, DELETE para borrar, etc. La mayoría de las APIs se quedan aquí.
- Nivel 3 (Controles Hipermedia): Nos volvemos RESTful de verdad. La API devuelve el recurso y, además, los enlaces para interactuar con él.

Si solo usamos JSON y URLs bonitas, somos API REST. Si seguimos el modelo hasta el nivel 3, tenemos una API RESTful.

Crecimiento de la web API: Con el **crecimiento de la web** y la popularización de aplicaciones web y móviles, el uso de servicios API para permitir la integración entre aplicaciones se volvió más común. Las **empresas comenzaron a exponer sus servicios y datos a través de APIs** públicas y privadas para permitir la integración con otras aplicaciones y servicios.

Adopción de REST: A medida que las **aplicaciones web y móviles se volvieron más comunes**, la arquitectura **REST se convirtió en un enfoque popular para diseñar servicios API** debido a su simplicidad, escalabilidad y facilidad de uso en entornos distribuidos.

Microservicios y Eficiencia: El auge del desarrollo de microservicios fue un motor muy importante en el crecimiento del desarrollo de servicios API, como así también la eficiencia en la comunicación entre aplicaciones.

Los servicios de API (Interfaz de Programación de Aplicaciones, por sus siglas en inglés) son conjuntos de **reglas** y **protocolos** que permiten que diferentes aplicaciones se comuniquen entre sí. Las API definen los **métodos** y **datos** que las aplicaciones pueden usar para solicitar y **compartir información**, lo que facilita la **integración** de sistemas y la creación de aplicaciones más complejas.

Una API especifica cómo interactuar con un servicio o sistema para realizar ciertas acciones o acceder a cierta información. Por ejemplo, API de redes sociales podría permitir a una aplicación publicar mensajes en una cuenta de usuario.

Las APIs pueden ser **públicas**, accesibles para cualquier desarrollador, o **privadas**, utilizadas solo dentro de una organización o por un grupo específico de desarrolladores.

Se pueden definir por:

- Su nivel de abstracción
- Su ubicación
- Su función
- Su estilo de arquitectura
- Su formato de datos

## Por su nivel de abstracción:

- API de sistema operativo: Permiten a las aplicaciones interactuar con el sistema operativo subyacente, como las API de Windows o las API de macOS.
- API de programación: Proporcionan funciones y procedimientos que pueden ser utilizados por los desarrolladores para construir aplicaciones, como las API de Java o las API de Python.

## Por su ubicación:

- API locales: Se utilizan para la comunicación entre componentes dentro de un mismo sistema, como las API de Java para interactuar con las bibliotecas estándar.
- API remotas: Se utilizan para la comunicación entre componentes que se ejecutan en sistemas diferentes, como las API web o las API de servicios web.



## Por su función:

- API web: Permiten a las aplicaciones interactuar con servicios web a través de Internet, utilizando protocolos como HTTP / HTTPS.
- API de bases de datos: Permiten a las aplicaciones acceder y manipular datos en una base de datos, como las API de MySQL o las API de MongoDB.
- API de hardware: Permiten a las aplicaciones interactuar con dispositivos de hardware, como las API de impresoras o las API de cámaras.

## Por su estilo de arquitectura:

- API REST: Utilizan el estilo arquitectónico REST (Representational State Transfer) para definir cómo las aplicaciones pueden acceder a recursos y realizar operaciones sobre ellos.
- API SOAP: Utilizan el protocolo SOAP (Simple Object Access Protocol) para definir cómo las aplicaciones pueden intercambiar mensajes XML estructurados.

## Formato de datos:

- API REST: Suelen utilizar formatos de datos ligeros y fáciles de leer, como **JSON** (+ liviano), para representar la información que se envía y se recibe.
- API SOAP: Utiliza exclusivamente **XML** (+ Pesado) para estructurar los mensajes, lo que puede hacer que los mensajes SOAP sean más pesados y difíciles de leer en comparación con JSON.

## Protocolo:

- API REST: Utiliza HTTP/HTTPS nativo. aprovecha los estándares ya existentes y ampliamente adoptados en Internet, lo que simplifica el desarrollo, mejora el rendimiento y garantiza la compatibilidad entre sistemas
- API SOAP: Utiliza HTTP/HTTPS, SMTP

## JSON

```
{
  "siblings": [
    {"firstName": "Anna", "lastName": "Clayton"},
    {"firstName": "Alex", "lastName": "Clayton"}
  ]
}
```

## XML

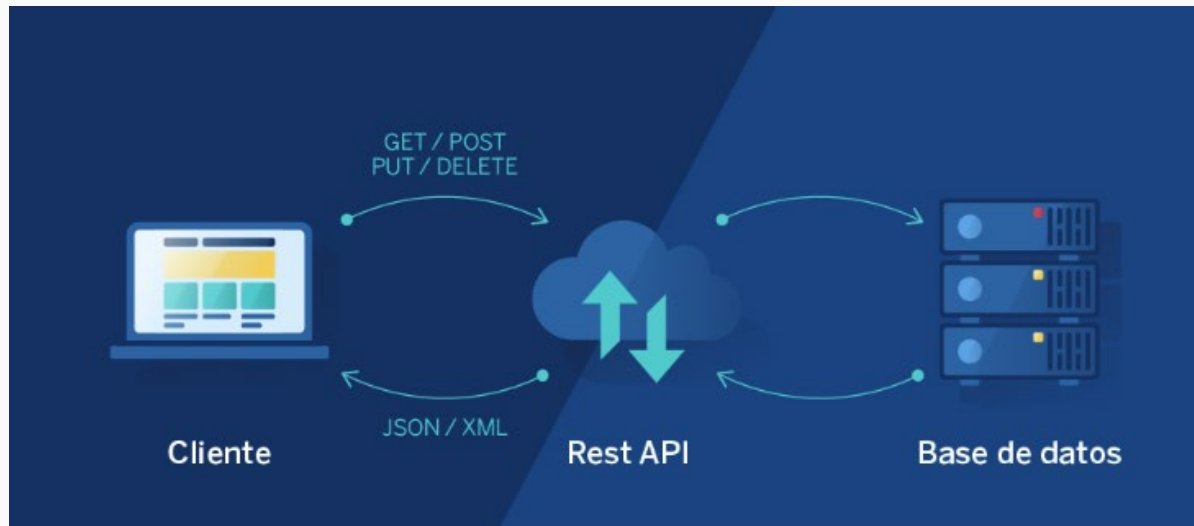
```
<siblings>
<sibling>
<firstName>Anna</firstName>
<lastName>Clayton</lastName>
</sibling>
<sibling>
<firstName>Alex</firstName>
<lastName>Clayton</lastName>
</sibling>
</siblings>
```

## Protocolo de comunicación:

- API REST: Utiliza el **protocolo HTTP** de forma nativa, lo que **facilita su implementación** y uso en entornos web.
- API SOAP: Utiliza un protocolo específico basado en **mensajes XML**, que puede requerir bibliotecas adicionales y configuraciones específicas para su uso en entornos web.

## Flexibilidad y simplicidad:

- API REST: Suele considerarse más **flexible y simple** de usar, ya que utiliza los verbos estándar de HTTP y sigue el principio de tener una interfaz uniforme.
- API SOAP: Puede considerarse más **compleja y rígida** debido a la estructura y las reglas estrictas definidas por el protocolo SOAP.



- GET
- POST
- PUT
- DELETE
- PATCH

```
[
  {
    "nombre": "a-Op",
    "nivel": {
      "nombre": "Nivel Primario"
    },
    "institucion": {
      "cve": "02000000",
      "nombre": "Escuela de Prueba",
      "distrito": "9"
    },
    "materias": [
      {
        "nombre": "Castellano"
      }
    ],
    "estudiantes": [
      {
        "dni": "80000001"
      },
      {
        "dni": "80000002"
      }
    ],
    "docentes": [
      {
        "dni": "18222999"
      }
    ]
  },
  {
    "nombre": "a-Op",
    "nivel": {
      "nombre": "Nivel Superior"
    },
    "institucion": {
      "cve": "02000000",
      "nombre": "Escuela de Prueba",
      "distrito": "9"
    },
    "materias": [
      {
        "nombre": "Castellano"
      }
    ],
    "estudiantes": [
      {
        "dni": "80000001"
      },
      {
        "dni": "80000003"
      },
      {
        "dni": "80000004"
      }
    ],
    "docentes": [
      {
        "dni": "14222333"
      },
      {
        "dni": "15444455"
      }
    ]
  }
]
```

**GET:** El método GET se utiliza para **recuperar información** de un recurso en el servidor. Al hacer una solicitud GET a una URL específica, el servidor devuelve la representación actual del recurso solicitado. Este método se utiliza comúnmente para **leer datos** y no debe tener ningún efecto secundario en el servidor, es decir, **no debe cambiar el estado de los datos**.

**POST:** El método POST se utiliza para enviar datos al servidor para crear un nuevo recurso. Este método se utiliza comúnmente para crear nuevos registros o enviar datos a un servidor para su procesamiento.

**PUT:** El método PUT se utiliza para **actualizar un recurso existente** en el servidor. Este método se utiliza comúnmente para **actualizar registros existentes** con datos nuevos o modificados.

**DELETE:** El método DELETE se utiliza para **eliminar un recurso existente** en el servidor. Este método se utiliza comúnmente para **eliminar registros o recursos que ya no son necesarios**.

**PATCH:** El método PATCH se utiliza **para actualizar parcialmente un recurso en el servidor**. Por ejemplo para modificar registros en situaciones complejas, como cuando se realizan actualizaciones que no requieren interacción del usuario o se usan formularios que abarcan varias pantallas.

## PATCH: ejemplo

```
{  
  "id": 123,  
  "nombre": "Juan Pérez",  
  "email": "juan@correo.com",  
  "bio": "Desarrollador de software aficionado al café",  
  "avatar": "foto_juan.jpg",  
  "puntos": 550  
}
```

Ruta: PATCH /usuarios/123

Cuerpo de la petición (JSON):

JSON



```
{  
  "bio": "Senior Dev y amante del té"  
}
```

**CRUD:** Las operaciones CRUD son las cuatro funciones fundamentales de la gestión de datos y bases de datos: Crear, Read (Leer), Update (Actualizar) y Delete (Borrar). Permiten la interacción básica con la información en cualquier aplicación, como registrar usuarios, ver perfiles, editar información o eliminar registros.

## 1. Autenticación y Autorización segura:

- OAuth2 / JWT (JSON Web Tokens): Utiliza tokens seguros para manejar la autenticación y autorización. OAuth2 permite que los usuarios autoricen a las aplicaciones a actuar en su nombre sin compartir sus credenciales.
- Autenticación Multifactor (MFA): Implementa MFA para aumentar la seguridad, especialmente en operaciones críticas.
- Scopes y Roles: Define roles y alcances (scopes) para limitar el acceso de los usuarios a ciertos recursos o acciones.

Un **token** es un elemento de seguridad digital que se utiliza para verificar la identidad de un usuario, autorizar acciones o proteger datos dentro de un sistema. Existen distintos tipos y usos.

### A. Token de Autenticación

- Se utiliza para identificar que un usuario ha iniciado sesión correctamente.
- Después de hacer login, el sistema entrega un token al navegador o aplicación.
- Este token se envía con cada solicitud al servidor para verificar que el usuario está autenticado.
- Ejemplo común: Token JWT (JSON Web Token).



## B. Token de Acceso

- Similar al anterior, pero se enfoca en dar permisos temporales para acceder a ciertos recursos.
- Se usa mucho en APIs. En lugar de enviar usuario y contraseña cada vez, se envía un token.
- Ejemplo: OAuth 2.0 Access Token.

## C. Token CSRF (Cross Site Request Forgery)

- Se usa para evitar que sitios maliciosos hagan solicitudes fraudulentas en nombre del usuario.
- El token se genera en el servidor y se incluye en los formularios HTML.
- El servidor solo acepta la solicitud si el token es válido.

## D. Token de Pago

- En sistemas de pagos (como en tarjetas de crédito), se reemplaza la información sensible por un token.
- Así, los datos reales no se almacenan ni se transmiten directamente.
- Ejemplo: Tokenización en plataformas como MercadoPago o Stripe.

## E. Token como Elemento Básico (en programación)

- En análisis léxico (como en compiladores), un token es la unidad más pequeña del lenguaje, como una palabra clave, operador, número, etc.

## ¿Qué contiene un Token de Autenticación (ej. JWT)?

Un token puede incluir:

- ID de usuario
- Tiempo de expiración
- Roles o permisos
- Firma digital para garantizar integridad

## ¿Por qué se usan tokens?

- Para evitar tener que enviar el usuario y contraseña en cada solicitud.
- Para mantener sesiones seguras en aplicaciones web o móviles.
- Para facilitar la comunicación entre servicios (APIs).
- Para aplicar expiración automática y revocación de acceso.

## Conclusión

- Un **token** es como una llave temporal que el sistema te da para que puedas abrir puertas específicas (acceder a recursos, hacer acciones, etc.) sin mostrar tus credenciales todo el tiempo.

## Token

Elemento de seguridad digital que verifica la identidad de un usuario, autoriza acciones o protege datos.



### Tipos y usos comunes:

- Token de autenticación
- Token de acceso
- Token CSRF
- Token de pago

### Qué contiene?

- ID de usuario
- Tiempo de expiración
- Firma digital



### Ejemplo de JWT:

Header.Payload.Signature

Clave temporal que se usa para acceder a recursos o realizar acciones de manera segura.

## 2. Comunicación Segura

- **HTTPS:** Asegúrate de que todas las comunicaciones entre el cliente y el servidor se realicen a través de HTTPS para encriptar los datos y protegerlos de ataques de interceptación (Man-in-the-Middle).
- **TLS:** Usa las versiones más recientes de TLS (Transport Layer Security) para proteger la transmisión de datos.

**HTTPS** (HyperText Transfer Protocol Secure) es la versión segura del protocolo HTTP. Se utiliza para la comunicación segura a través de una red, principalmente en Internet.

Utiliza SSL/TLS (Secure Sockets Layer / Transport Layer Security) para cifrar los datos que se envían entre el navegador del usuario y el servidor web, protegiéndolos contra espionaje o manipulación.

### A. Cifrado de datos

- Toda la información intercambiada está cifrada, lo que impide que terceros puedan leer o modificar los datos (por ejemplo, contraseñas o tarjetas de crédito).

### B. Autenticación

Usa certificados digitales emitidos por una Autoridad de Certificación (CA) para asegurar que el sitio web es auténtico y no una copia falsa.

## **C. Integridad**

- Asegura que los datos no se alteraron durante la transmisión. Si se detecta una modificación, la comunicación se interrumpe.

## **D. Indicador en navegadores**

- La mayoría de los navegadores muestran un candado al lado de la URL cuando se usa HTTPS, lo que da confianza al usuario.

## **E. Requerido para SEO y e-commerce**

- Google favorece los sitios con HTTPS en sus resultados de búsqueda.
- Es imprescindible en plataformas que manejan datos sensibles (login, pagos).



## Diferencias entre HTTP y HTTPS

| Característica      | HTTP           | HTTPS            |
|---------------------|----------------|------------------|
| Seguridad           | No segura      | Segura (cifrada) |
| Puerto              | 80             | 443              |
| Certificado SSL     | No requerido   | Requerido        |
| Cifrado             | No             | Sí (SSL/TLS)     |
| Integridad de datos | No garantizada | Garantizada      |



## 3. Validación de Datos

- Validación del Lado del Servidor: Valida todas las entradas en el servidor para proteger contra ataques de inyección (como SQL injection o XSS).
- Saneamiento de Entradas: Limpia y filtra los datos entrantes para evitar que datos maliciosos se procesen.

## 4. Manejo Adecuado de Errores

- Códigos de Estado HTTP Correctos: Devuelve códigos de estado HTTP apropiados (como 200 OK, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error) para que los clientes puedan manejar los errores correctamente.
- Mensajes de Error Generales: No expongas detalles internos del servidor en los mensajes de error; mantén los mensajes genéricos y registra los detalles en el servidor.

## 5. Rate Limiting y Control de Tráfico

- Rate Limiting: Implementa límites de frecuencia para prevenir abusos y ataques de denegación de servicio (DDoS).
- Throttling (estrangulamiento): Restringe el número de solicitudes que un usuario o IP puede hacer en un período de tiempo específico.

## 6. CORS (Cross-Origin Resource Sharing)

- Configuración CORS: Controla qué dominios pueden hacer solicitudes a tu API para protegerla de ataques como Cross-Site Request Forgery (CSRF).

## 7. Logs y Auditoría

- Registro de Actividad: Mantén un registro de todas las actividades importantes (como accesos y modificaciones) en la API para auditar y detectar comportamientos anómalos.
- Monitoreo: Utiliza herramientas de monitoreo para detectar y alertar sobre comportamientos inusuales o intentos de ataque en tiempo real.

## 8. Seguridad en las Claves API

- Uso de Claves API: Si utilizas claves API, asegúrate de que estén cifradas y almacenadas de manera segura. Proporciona a los desarrolladores claves individuales con permisos limitados.
- Rotación de Claves: Implementa políticas de rotación de claves API para que las claves comprometidas puedan ser reemplazadas rápidamente.



## 9. Pruebas de Seguridad

- **Pentesting Regular:** Realiza pruebas de penetración periódicas para identificar vulnerabilidades de seguridad.
- **Pruebas Automatizadas:** Implementa pruebas automatizadas de seguridad en el pipeline de CI/CD para detectar vulnerabilidades antes de cada despliegue.

## 10. Documentación Clara y Detallada

- **Documentación de Seguridad:** Incluye en la documentación de la API las mejores prácticas de seguridad que los usuarios deben seguir, como el manejo adecuado de tokens y la configuración correcta de CORS.

**Al implementar estas características, ASEGURAMOS que nuestras API REST no solo sean funcional, sino también segura y alineada con las mejores prácticas del desarrollo web moderno.**



¿Por qué probar APIs o servicios?

En contextos de desarrollo de software ágil, probar lo antes posible:

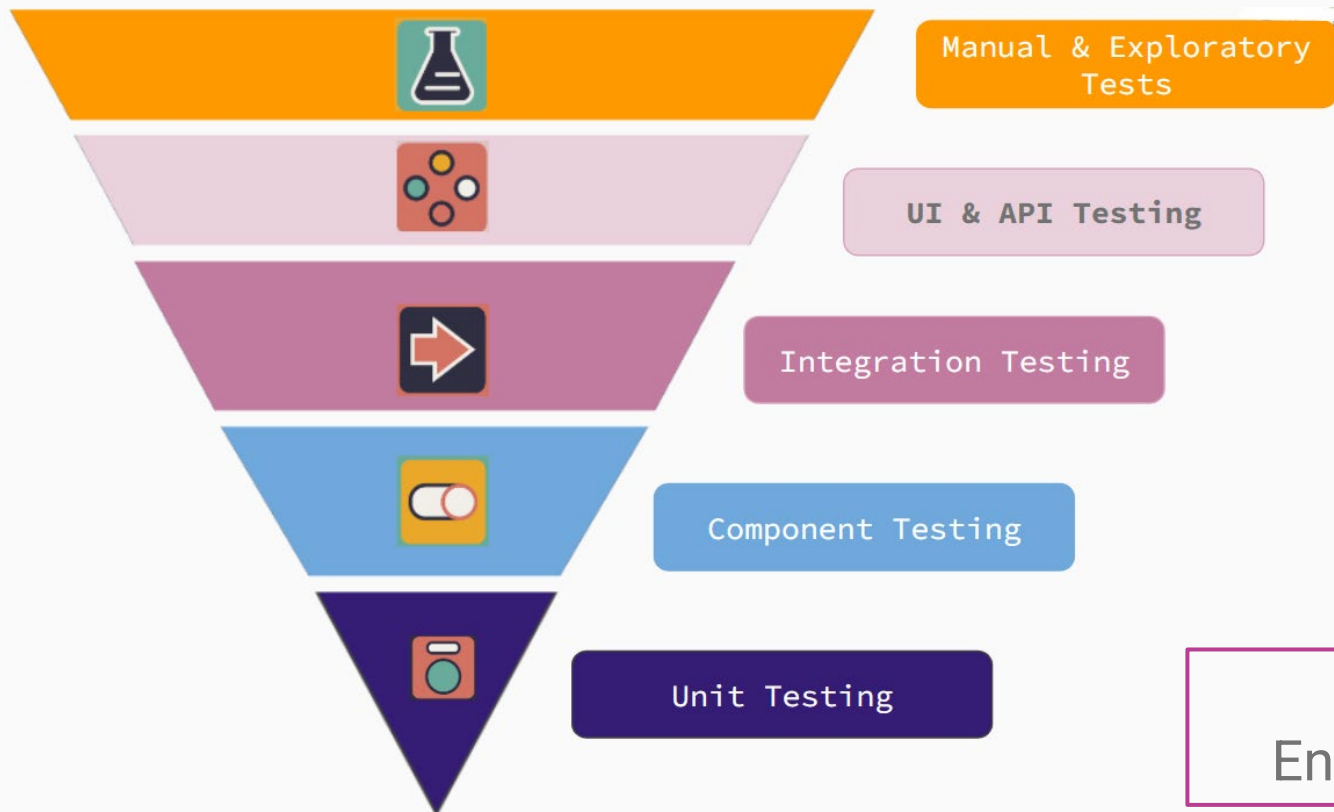


**Shift - left - testing**

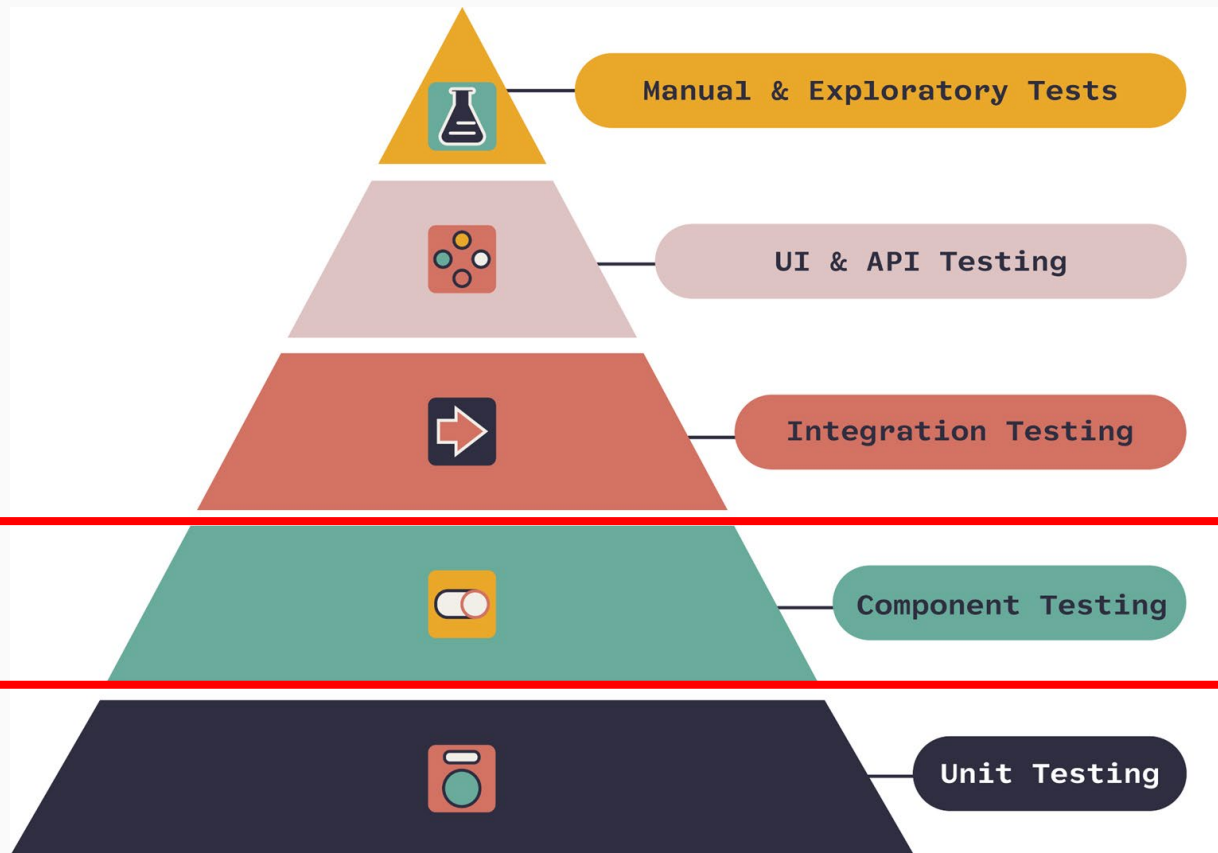


Los servicios son software:

**Deben probarse**



Objetivo:  
Encontrar errores



Objetivo:  
**Prevenir errores**

## Integradas a la aplicación cliente:

- Ventaja: Podemos ver el comportamiento final
- Desventaja: si hay errores en las APIs, se demora la detección del mismo hasta tenerla integrada a algún cliente.

## Antes de integrarlas a las aplicaciones cliente:

- Ventaja: pruebas tempranas, detectan errores en forma temprana, llegan al módulo cliente depuradas
- Desventaja: la prueba manual a veces no es trivial

## Herramientas de prueba de API:

- **Postman:** Es una herramienta popular para probar servicios API. Permite enviar solicitudes HTTP a una API, ver las respuestas y validar los resultados.
- **Insomnia:** Similar a Postman, Insomnia es una herramienta de prueba de API que te permite enviar solicitudes HTTP y ver las respuestas de forma interactiva.
- **Swagger UI:** Si la API está documentada con OpenAPI (anteriormente conocido como Swagger).

## Cliente HTTP en el navegador:

- Extensiones de navegador: Puedes utilizar extensiones como Postman Interceptor (para Postman) o RESTClient (para Firefox) para enviar solicitudes HTTP y ver las respuestas directamente desde tu navegador.

## Pruebas automatizadas:

- Frameworks de pruebas: Puedes utilizar frameworks de pruebas como PHPUnit (para PHP), RestAssured y RestApi (para Java), Frisby (para Node.js), o frameworks más completos, que integren alguna de las mencionadas antes, como Karate.

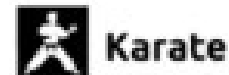
## Herramientas de línea de comandos:

- cURL: Es una herramienta de línea de comandos que te permite enviar solicitudes HTTP directamente desde la terminal.
- HTTPie: Es una herramienta similar a cURL pero con una sintaxis más amigable y fácil de recordar.

## Entornos de desarrollo integrados (IDE):

- Algunos IDEs, como IntelliJ IDEA, Visual Studio Code o Eclipse, tienen extensiones o plugins que facilitan la prueba de servicios API REST directamente desde el entorno de desarrollo.





POSTMAN

- Conseguir la documentación
  - El concepto de exploración NO APLICA
  - Si o si debo tener la documentación en algún formato
- Entender la documentación
  - Dudas con el responsable del diseño (preferentemente no un desarrollador)
  - Limitaciones que puedo encontrar
- Planificar las pruebas:
  - Identificar endpoints que voy a probar
  - Escribir los casos de prueba
- Herramienta de prueba:
  - ¿Donde voy a probar?
  - Preferentemente herramienta que me permita dejar almacenadas las peticiones para futuras repeticiones.
- Ejecutar pruebas, capturar evidencias y reportar errores.

- Postman es una herramienta para probar web services e interfaces de aplicación (API) en arquitectura REST.
- Permite enviar solicitudes a cualquier servidor web y proporciona y cataloga la respuesta.
- Comenzó como un plugin del navegador Chrome, pero luego fue lanzada como versión nativa tanto para Windows como Mac.
- Posee una versión gratuita, versión profesional y versión empresarial a diferentes costos.
- Puede descargarse e instalarse desde aquí (o puede usarse la versión web):

<https://www.postman.com/downloads/>



Es tiempo de  
**PREGUNTAS**

GRACIAS

